XM · CLASE 8

Soluciones de Analítica con Databricks

Clase 8: Código, configuración y datos separados

Docente: Manuela Larrea Gómez

Agenda de la clase 8

Más teoría que de costumbre, con receso en la mitad; el notebook aplica todo sobre tu propio repo

01	25 MIN	**Teoría, parte 1** Por qué separar · notebooks delgados + paquete · el camino de un cambio · la pirámide de pruebas
02	10 MIN	**Receso**
03	20 MIN	**Teoría, parte 2** ruff y pre-commit · configuración sin secretos · asistentes de código con verificación · reproducibilidad ante auditoría
04	55 MIN	**Notebook guiado + lab 8** validar_config · contrato de oro como código · pytest y ruff desde Databricks · test de la fuga · el PR que debe fallar
05	10 MIN	**Cierre** Hito 4 cerrado · tag s08 · así sería en Azure · próxima clase: MLflow

Código, configuración y datos: tres ritmos de cambio, tres manos

Mezclados, cada cambio en uno obliga a tocar los otros dos. Separados, cada uno tiene dueño, ritmo y forma de revisarse

Código, configuración y datos: tres cosas que cambian a ritmos distintos y por manos distintas

Si viven mezclados, cada cambio en uno obliga a tocar los otros dos. Separados, cada uno tiene su dueño, su ritmo y su forma de revisarse.

CÓDIGO · `src/xm_demanda/`

Qué se hace y cómo: ingesta, reglas, pivote, features. Cambia cuando cambia la **lógica**. Se revisa con PR, lint y tests. Es el mismo en dev, qa y prod.

+

CONFIGURACIÓN · `conf/*.yml`

Dónde y con qué parámetros: catálogo, volúmenes, lags, horizonte. Cambia cuando cambia el **ambiente o el experimento**. Sin secretos: esos van al secret scope.

+

DATOS · Unity Catalog

Lo que llega y lo que se construye: bronce, plata, oro, referencia. Cambia **todos los días**. Se gobierna con permisos, versiones Delta y expectations; nunca se edita a mano.

código: cambia por semanas · lo cambia ingeniería · lo revisa CI | **configuración: cambia por ambiente o experimento · lo cambia quien despliega · lo revisa un PR** | **datos: cambian a diario · los cambia la fuente · los revisan las expectations**

Pregunta	Mezclado (el notebook que lo tiene todo)	Separado (este repo)
Pasar a prod	Copiar el notebook, buscar y cambiar `dev` por `prod` en 14 sitios, rezar	`load_config("prod")`: mismo código, otro YAML
Probar la regla de calidad	Correr el notebook completo contra datos reales (20 min, y sin saber si pasó)	`pytest tests/unit/test_rules.py`: 0,1 s, sin Spark
Cambiar el horizonte a 14 días	Editar código en tres notebooks	Un número en `conf/base.yml`; el contrato de oro lo sigue
Rotar una contraseña	Buscarla en el repo (y en el historial de Git, para siempre)	Cambiarla en el secret scope; el repo no la conoció nunca

Desde la clase 4 trabajan así. Hoy lo hacemos explícito y lo probamos: validar_config, el contrato como código y CI con integración.

Fuente: Databricks, Software engineering best practices for notebooks · The Twelve-Factor App, III. Config (12factor.net/config)

Notebooks delgados + paquete: el mapa del repo tal como está hoy

Nada nuevo que construir; sí que entender por qué cada cosa vive donde vive

Capa	Dónde	Qué contiene	Cómo se prueba
Notebooks de producción	notebooks/01…06	Widget catalog → load_config → llama a src. Sin lógica	Corren como tareas de un job (clase 12)
Paquete	src/xm_demanda/ {ingest,quality,silver,features,utils}	Una función por responsabilidad; las puras sin Spark	tests/unit (0,1 s) y tests/integration (Spark, 20 s)
Configuración	conf/base.yml + conf/{dev,qa,prod}.yml	Catálogo, volúmenes, tablas, lags, horizonte. Sin secretos	validar_config + test_config.py (hoy)
Datos de referencia	data/reference/*.csv	Festivos, CIIU: cambian poco, se revisan por PR	verificar() cuando un año no está cubierto
Labs	clases/cNN/notebooks	Teoría + llamadas a src + verificar()	verificar() sobre tus datos reales
Decisiones	docs/adr	Una ADR por decisión; se compara, no se califica	Revisión en el PR

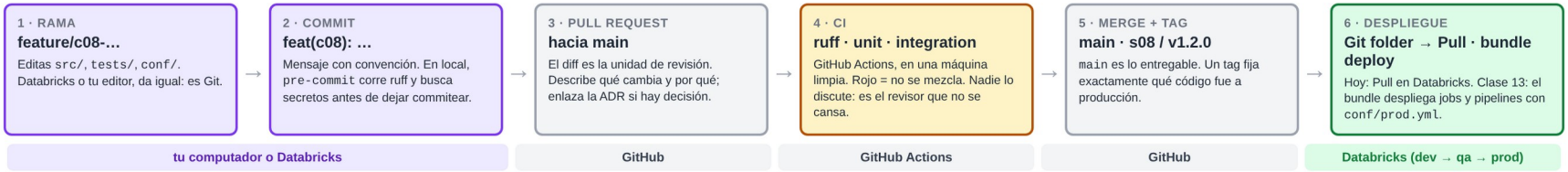
Regla del curso: el notebook llama, src implementa, conf parametriza, tests prueban. Si una celda tiene un if de negocio, va a src.

El camino de un cambio: de tu rama a la tabla de producción

El mismo camino desde la clase 4; hoy con un job más en CI y un PR que debe fallar

El camino de un cambio: de tu rama a la tabla de producción

Es el mismo camino desde la clase 4. Hoy lo miramos completo, con lo que pasa en cada parada y qué lo detiene.



Lo que hoy vas a provocar a propósito: un PR que cambia `rowsBetween(-7, -1)` por `(-6, 0)`. ruff pasa, las unitarias pasan, y la integración falla en `test_media_7_nunca_incluye_el_dia_t`. Ese rojo es el sistema funcionando: una fuga de información detenida antes de tocar un dato.

Hoy en el lab

PR 1: feature/c08-tests-ci con los tres TODO → verde → merge. PR 2: feature/c08-fuga con rowsBetween(-n + 1, 0) → integration en rojo → captura → cerrar sin mezclar.

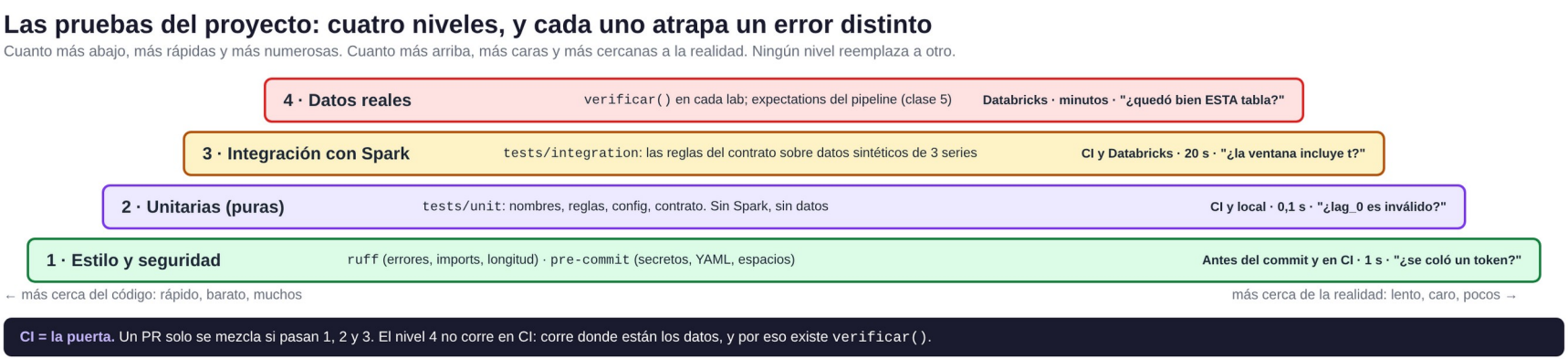
En XM

main protegida: no se mezcla sin checks en verde y un revisor. Un tag v1.2.0 dispara el despliegue a qa y, con aprobación manual, a prod (clase 13).

Fuente: docs.github.com → About protected branches · docs.databricks.com → Git folders

Las pruebas del proyecto: cuatro niveles, cuatro errores distintos

Abajo: rápidas, baratas, muchas. Arriba: lentas, cercanas a la realidad, pocas. Ninguna reemplaza a otra



Nivel	Ejemplo real de hoy	Qué NO puede ver
ruff / pre-commit	Línea de 108 caracteres; token en un YAML	Que la función devuelva mal
Unitarias	nombres_lags([0]) → ValueError; columnas_contrato sigue al YAML	Que la ventana de Spark incluya t
Integración (Spark)	media_7 ≠ avg(rowsBetween(-6, 0)); lag_7 = 7 filas atrás	Que XM republicó marzo
verificar() / expectations	72.704 filas; es_festivo sin nulos en 2026	— (es la realidad)

RECESO

10 minutos

Al volver: ruff y pre-commit · configuración sin secretos · asistentes de código · reproducibilidad ante auditoría

Mientras tanto: Git folder en main → Pull, y rama feature/c08-tests-ci creada

iData
Cátedra iData & AI

Antes del commit: ruff, pre-commit y las convenciones del repo

Ya están configurados en pyproject.toml, .pre-commit-config.yaml y CONTRIBUTING.md

ruff

Linter + formateador, en Rust: ~1 s sobre todo el repo. Reglas E, F, I, B, UP; línea de 100. Lo que CI rechaza primero. Desde Databricks: python -m ruff check src tests.

pre-commit

Hooks que corren antes de que el commit exista: ruff, YAML válido, espacios finales, llaves privadas y gitleaks (secretos). En local: pip install -e ".[dev]" && pre-commit install.

Convenciones

snake_case y unidades en el nombre; feature/cNN-tema; Conventional Commits (feat, fix, test, docs); una ADR por decisión; el notebook llama, src implementa.

terminal

```
$ ruff check src
src/xm_demanda/features/build.py:85:101: E501 Line too long (108 > 100)
$ git commit -m "feat: token de prueba"      # con pre-commit instalado
gitleaks.................Failed   Finding: token = "ghp_6xK..."  →  el commit no se crea
$ git commit -m "feat(c08): validar_config, contrato de oro y test de fuga en integracion"   ✓
```

Fuente: docs.astral.sh/ruff · pre-commit.com · github.com/gitleaks/gitleaks · conventionalcommits.org

Configuración por ambiente, sin secretos

base.yml tiene lo que no cambia; <env>.yml solo lo que cambia. Las credenciales nunca pasan por el repo

```
conf/
# conf/base.yml
schemas: {bronze: bronze_energia, silver: …, gold: …}
modelo: {horizon_days: 7, lags: [1, 7, 14, 28], …}
# conf/prod.yml
catalog: prod
volumen_landing: /Volumes/prod/bronze_energia/landing
secreto_fuente: {scope: xm-prod, key: api_xm_token}
# ↑ el NOMBRE del secreto, nunca el valor
```

```
src/ + secret scope
# src/xm_demanda/utils/config.py (TODO 1)
cfg = load_config(catalog)     # base + ambiente
validar_config(cfg)
#  falta 'modelo'   → ValueError('... falta ...')
#  'db_password'    → ValueError('... secreto ...')
# en ejecución, si hiciera falta:
token = dbutils.secrets.get(scope, key)
print(token)                   # [REDACTED]
```

Dónde	Qué va	Qué NO va
conf/*.yml (repo)	catálogo, rutas, tablas, parámetros del modelo, nombres de secretos	contraseñas, tokens, cadenas de conexión
Secret scope (Databricks) / Key Vault (Azure)	el valor del secreto, con ACL por grupo	parámetros del modelo (no son secretos; deben verse en el PR)

Fuente: docs.databricks.com → Secret management · 12factor.net/config

Asistentes de código: para escribir tests y explicar errores, no para decidir

Databricks Assistant está en el notebook y en el editor; la regla del repo no cambia por eso

Para qué sirven hoy

Proponer un test a partir de una función de src (luego lo lees y lo rompes a propósito). Explicar un error de Spark en español. Sugerir el docstring. Convertir una consulta SQL a PySpark.

Dónde fallan con total seguridad

rowsBetween(-6, 0) 'para una media móvil de 7 días'. Un coalesce(False) que esconde el año sin festivos. Un MERGE sin condición de versión. Todo compila, todo se ve bien.

Regla	Cómo se cumple en este repo
Lo propuesto es un borrador	Entra por PR como cualquier cambio; el diff se lee línea a línea
El test decide, no la confianza	Un test que no puede fallar no prueba nada: por eso el segundo assert de hoy
Verificar sobre datos	verificar() y las expectations: la realidad tiene la última palabra
Explicar es más valioso que generar	Pedir 'por qué falla' antes que 'arréglalo'; lo que no entiendes no lo mezclas

Fuente: docs.databricks.com → Databricks Assistant FAQ (qué datos usa, cómo verificar sugerencias)

Sector: reproducibilidad como requisito ante auditoría regulatoria

Reconstruir hoy, exactamente, el pronóstico que se publicó hace seis meses

Reproducibilidad: qué hace falta para reconstruir hoy el pronóstico que se publicó hace seis meses

Ante una auditoría del regulador no basta con "el modelo dio eso". Hay que poder volver a producir exactamente el mismo número y explicar cada paso.

Un pronóstico = código × configuración × datos × modelo. Los cuatro tienen versión. Si falta una, no es reproducible.

Código

`git tag v1.2.0 · commit 8f3a…`

Qué funciones corrieron. El PR, el CI en verde y el diff viven en GitHub para siempre.

×

Configuración

`conf/prod.yml @ 8f3a…`

Lags, ventanas, horizonte, catálogo. Va en el mismo commit que el código: una sola versión.

×

Datos

`features_demanda_diaria VERSION AS OF 41`

Time travel de Delta (clase 6). La versión de la tabla de oro que leyó el entrenamiento y la inferencia.

×

Modelo

`MLflow run 2c9e… · models.demanda_7d@champion`

Parámetros, métricas, artefacto y, dentro del run, los tres identificadores de la izquierda (clase 9).

Hoy ya tienes tres de los cuatro: el tag `sNN` de cada clase, la configuración versionada con el código y el historial Delta de cada tabla. El cuarto llega con MLflow en la clase 9.

Lo que lo rompe: lógica en un notebook sin commit, un parámetro cambiado a mano en el job, `VACUUM` con retención corta sobre la tabla de entrenamiento, un modelo entrenado "en mi máquina".

Un pronóstico que no se puede reconstruir no se puede defender. XM reporta a la CREG y a la SSPD; sus agentes le reportan a XM. Cada número tiene que tener un origen trazable.

3 de 4

versiones ya las tienen: tag, config en el commit, historial Delta

7 días

retención por defecto de VACUUM: con eso no hay seis meses (pregunta 2 del lab)

clase 9

MLflow fija la cuarta: el run guarda commit, config y versión de datos

Fuente: ISO/IEC 5338:2023 (ciclo de vida de sistemas de IA) · creg.gov.co · superservicios.gov.co · Databricks, The Big Book of MLOps

Free Edition vs. Azure Databricks: qué simulamos hoy

Los tests y el contrato son los mismos; cambia el motor de CI, de dónde salen los secretos y quién mezcla

Elemento	Hoy en Free Edition	En XM sobre Azure
CI	GitHub Actions en tu fork: test + integration (Spark local)	Azure DevOps (ci/azure-pipelines.yml) o GitHub Actions; los mismos jobs
Tests desde Databricks	%pip install pytest ruff y pytest.main() en el notebook del lab	Igual; además, tests de integración en dev vía bundle (clase 13)
Secretos	No hacen falta (dataset en el repo)	Key Vault enlazado a un secret scope; ACL por grupo; rotación sin tocar el repo
Protección de main	Opcional (extensión del lab)	Obligatoria: checks en verde + revisor; nadie hace push directo
Asistente de código	Databricks Assistant en el notebook	Igual, con políticas del workspace sobre qué datos puede ver

Notebook y lab 8: tres TODO, un PR en verde y un PR en rojo

clases/c08-codigo-config/notebooks/lab08_tests_ci · widget usuario · requiere features_demanda_diaria (lab 7)

1. **Secciones 1–3: config y contrato**
   validar_config (TODO 1) y columnas_contrato (TODO 2) en src
2. **Sección 4: ruff y pytest desde Databricks**
   Lo mismo que hace CI; los skip son TODO pendientes
3. **Sección 5: test_media_7_nunca_incluye_el_dia_t**
   TODO 3, con dos assert: el correcto y el que distingue la fuga
4. **Sección 6: el PR que debe fallar**
   PR bueno en verde → PR con rowsBetween(-n + 1, 0) en rojo → captura → cerrar
5. **Sección 7 + verificar() · commit · PR**
   feat(c08): validar_config, contrato de oro y test de fuga en integracion

Criterios de aceptación (hito 4 cerrado)

- test_config.py y test_features.py sin skip
- tests/integration en verde, con tu test de la fuga
- Un PR mezclado con los 3 jobs en verde
- Un PR cerrado sin mezclar, con integration en rojo (captura)
- Las dos preguntas; verificar() en OK

Comparar con s08

github.com/<docente>/Databricks-to-XM/compare/s08...<tú>:Databricks-to-XM:main · o git diff s08 -- src/xm_demanda/utils src/xm_demanda/features tests .github/workflows

Duración: 25 min guiado + 30 min solo

Así sería en Azure Databricks: mismo repo, otro CI, otro origen de secretos

Hoy	En XM
GitHub Actions: test + integration en cada PR	Azure Pipelines con las mismas etapas (ci/azure-pipelines.yml); agentes propios si el dato no puede salir
Tests de integración con Spark local y datos sintéticos	Igual en CI; más una corrida en dev vía bundle con datos reales enmascarados (clase 13)
Secretos: ninguno	Key Vault → secret scope; la configuración solo nombra el secreto; rotación sin PR
main de tu fork, sin protección	main protegida: checks obligatorios, un revisor, sin push directo; tags v* despliegan
Reproducibilidad: tag + config + versión Delta	Lo mismo, más el run de MLflow y retención Delta ampliada en tablas de entrenamiento

Lo que no cambia: funciones puras con test, configuración validada sin secretos, el contrato de oro en código y un PR que puede ponerse rojo.

Resumen de la clase 8 y preparación para la clase 9

Logros de hoy

- **Separación, con nombre**
  Código en src, configuración en conf, datos en UC: cada uno a su ritmo
- **Config validada, sin secretos**
  validar_config: llaves requeridas y nada que parezca una credencial
- **Contrato como código**
  columnas_contrato sigue al YAML y verificar() lo compara con la tabla
- **CI que detiene errores**
  Job integration con Spark; el PR con fuga se puso rojo
- **Hito 4 cerrado**
  Paquete xm_demanda con tests en verde y evidencia de CI

Próxima clase

- **Experimentación con MLflow**
  Experimentos, runs, artefactos, log_input; comparar modelos
- **Baseline estacional-naive**
  lag_7 como pronóstico: difícil de vencer en demanda eléctrica
- **Modelo global LightGBM**
  Sobre features_demanda_diaria; métricas por segmento
- **Traer a la clase 9**
  Lab 8 con el PR en verde; la clase 9 entrena sobre tu tabla de oro





Gracias

Nos vemos en la clase 9: Experimentación con MLflow

DOCENTE

Manuela Larrea Gómez